

Отчёт по домашнему заданию

ClickHouse: репликация, шардирование, Distributed и TTL

Стенд: Yandex Cloud VM + Docker Compose, 4 экземпляра ClickHouse и 3 экземпляра ClickHouse Keeper

1. Краткое резюме

Параметр	Значение
Инфраструктура	Одна VM в Yandex Cloud, Docker Compose
ClickHouse-ноды	ch01, ch02, ch03, ch04
Кеерер-ноды	keeper01, keeper02, keeper03
Версия ClickHouse	26.3.12.3
Основная БД	otus
Реплицируемая таблица	otus.events_local
TTL	event_date + INTERVAL 7 DAY

Критерии выполнены: экземпляры ClickHouse запущены, репликация и шардирование настроены, Distributed-таблицы созданы, проверочные результаты сохранены в файлы.

2. Что было сделано

- На одной VM в Yandex Cloud был поднят учебный стенд через Docker Compose.
- Запущены четыре независимых экземпляра clickhouse-server: ch01, ch02, ch03, ch04.
- Запущены три экземпляра ClickHouse Keeper: keeper01, keeper02, keeper03. Keeper используется для координации ReplicatedMergeTree.
- В remote_servers.xml описаны несколько кластерных топологий с разным числом шардов и реплик.
- Созданы локальные и Distributed-таблицы для проверки распределённых запросов.
- Создана реплицируемая таблица с TTL, хранящим последние 7 дней данных от значения в колонке event_date.

3. Настроенные топологии

Кластер	Шардов	Фактор репликации	Состав
cluster_1s_4r	1	4	shard 1: ch01, ch02, ch03, ch04
cluster_2s_2r	2	2	shard 1: ch01, ch02; shard 2: ch03, ch04
cluster_4s_1r	4	1	shard 1: ch01; shard 2: ch02; shard 3: ch03; shard 4: ch04

```
SELECT
    cluster,
    shard_num,
    replica_num,
    host_name
FROM system.clusters
ORDER BY cluster, shard_num, replica_num;
```

Результат сохранён в файл system_clusters.txt.

4. Distributed-таблицы

Для проверки распределённых запросов создана локальная таблица на всех нодах:

```
CREATE TABLE IF NOT EXISTS otus.local_one ON CLUSTER cluster_2s_2r
(
    dummy UInt8
)
ENGINE = MergeTree
ORDER BY dummy;
```

В разные ноды были вставлены разные значения:

```
ch01: 1
ch02: 2
ch03: 3
ch04: 4
```

Затем были созданы Distributed-таблицы для разных топологий:

```
CREATE TABLE IF NOT EXISTS otus.dist_1s_4r ON CLUSTER cluster_2s_2r
(
    dummy UInt8
)
ENGINE = Distributed(cluster_1s_4r, otus, local_one, rand());

CREATE TABLE IF NOT EXISTS otus.dist_2s_2r ON CLUSTER cluster_2s_2r
(
    dummy UInt8
)
ENGINE = Distributed(cluster_2s_2r, otus, local_one, dummy);

CREATE TABLE IF NOT EXISTS otus.dist_4s_1r ON CLUSTER cluster_2s_2r
(
    dummy UInt8
)
ENGINE = Distributed(cluster_4s_1r, otus, local_one, dummy);
```

Проверочные запросы для каждой Distributed-таблицы:

```
SELECT
    *,
    hostName(),
    _shard_num
FROM otus.dist_1s_4r
ORDER BY hostName(), dummy;

SELECT
    *,
    hostName(),
    _shard_num
FROM otus.dist_2s_2r
ORDER BY _shard_num, hostName(), dummy;

SELECT
    *,
    hostName(),
    _shard_num
FROM otus.dist_4s_1r
ORDER BY _shard_num, hostName(), dummy;
```

Результаты сохранены в файлы `dist_1s_4r_result.txt`, `dist_2s_2r_result.txt` и `dist_4s_1r_result.txt`. Команды `SHOW CREATE TABLE` сохранены в `show_create_dist_*.sql`.

5. Репликация и TTL

Для проверки репликации создана таблица otus.events_local на движке ReplicatedMergeTree:

```
CREATE TABLE IF NOT EXISTS otus.events_local ON CLUSTER cluster_2s_2r
(
    event_date Date,
    id UInt64,
    value String
)
ENGINE = ReplicatedMergeTree(
    '/clickhouse/tables/{shard}/otus/events_local',
    '{replica}'
)
ORDER BY id
TTL event_date + INTERVAL 7 DAY;
```

В этой таблице используются макросы:

- {shard} - разделяет путь в Кеерер по шардам.
- {replica} - задаёт имя конкретной реплики.

Для распределённой вставки и чтения создана Distributed-таблица:

```
CREATE TABLE IF NOT EXISTS otus.events_dist ON CLUSTER cluster_2s_2r
AS otus.events_local
ENGINE = Distributed(cluster_2s_2r, otus, events_local, id);
```

Тестовые данные включали свежие и устаревшие строки:

```
INSERT INTO otus.events_dist VALUES
    (today(), 1, 'fresh row 1'),
    (today(), 2, 'fresh row 2'),
    (today() - 10, 3, 'old row 1'),
    (today() - 10, 4, 'old row 2');
```

Для проверки TTL был выполнен принудительный merge:

```
OPTIMIZE TABLE otus.events_local ON CLUSTER cluster_2s_2r FINAL;
```

После merge строки с датой today() - 10 удаляются согласно TTL event_date + INTERVAL 7 DAY.

6. Проверочные запросы и файлы

Проверка system.parts через remote:

```
SELECT
    getMacro('replica') AS replica,
    *
FROM remote(
    'ch01:9000,ch02:9000,ch03:9000,ch04:9000',
    system.parts
)
WHERE database = 'otus'
    AND table = 'events_local'
FORMAT JSONEachRow;
```

Результат сохранён в parts.json.

Проверка system.replicas:

```
SELECT *
FROM system.replicas
WHERE database = 'otus'
```

```
    AND table = 'events_local'
    FORMAT JSONEachRow;
```

Результат сохранён в *replicas.json*.

Дополнительная проверка всех реплик:

```
SELECT *
FROM clusterAllReplicas(
    cluster_2s_2r,
    system.replicas
)
WHERE database = 'otus'
    AND table = 'events_local'
    FORMAT JSONEachRow;
```

Результат сохранён в *replicas_all_nodes.json*.

7. Файлы для сдачи

Файл	Назначение
remote_servers.xml	XML-секция с топологиями кластеров
system_clusters.txt	Вывод system.clusters
show_create_dist_1s_4r.sql	DDL Distributed-таблицы для cluster_1s_4r
show_create_dist_2s_2r.sql	DDL Distributed-таблицы для cluster_2s_2r
show_create_dist_4s_1r.sql	DDL Distributed-таблицы для cluster_4s_1r
dist_1s_4r_result.txt	Результат SELECT из otus.dist_1s_4r
dist_2s_2r_result.txt	Результат SELECT из otus.dist_2s_2r
dist_4s_1r_result.txt	Результат SELECT из otus.dist_4s_1r
parts.json	Результат запроса к system.parts через remote
replicas.json	Результат запроса к system.replicas
replicas_all_nodes.json	Расширенная проверка system.replicas со всех реплик

8. Вывод

В ходе работы были запущены несколько экземпляров clickhouse-server, настроены разные топологии кластеров с шардами и репликами, созданы Distributed-таблицы для каждой топологии, а также создана реплицируемая таблица ReplicatedMergeTree с TTL-политикой хранения данных 7 дней.

Критерии выполнения задания выполнены: экземпляры ClickHouse настроены, репликация и шардирование созданы, Distributed-таблицы работают корректно, результаты проверочных запросов сохранены в отдельные файлы.